Strategic Guide on How to Approach a Low-Level Design (LLD) Interview

Low-Level Design (LLD) interviews can be challenging but are a crucial part of software engineering interviews. This article will serve as a step-by-step guide to approaching an LLD interview, addressing common doubts, and providing strategies to succeed. By following these steps, you will learn how to break down the problem, design the system efficiently, and communicate your thought process clearly. Whether you're preparing for an upcoming interview or just improving your design skills, this guide will help you navigate the complexities of LLD interviews with confidence.

Step 1: Clarify Requirements

In any Low-Level Design (LLD) interview, the first step is always to clarify the requirements. Before diving into the design, it’s crucial to understand what exactly needs to be built. This step will ensure you and the interviewer are on the same page and prevent you from building the wrong solution. The requirements can be broken down into functional and non-functional requirements, along with hero use cases.

A. Functional Requirements

Functional requirements define the core features and functionality of the system. These are the behaviors and processes that the system must perform. Essentially, these requirements answer the question: What should the system do?

For example:

- In an e-commerce system, functional requirements could include the ability to browse products, add items to the cart, and process payments.
- In a parking lot system, functional requirements could include the ability to park a car, remove a car, or check the parking space availability.

Key Points:

- Focus on main features.
- Mention any specific workflows.
- Identify constraints or business rules that need to be followed.

B. Non-Functional Requirements

Non-functional requirements are the qualities or characteristics of the system, rather than specific behaviors. They define how well the system performs its functions. These are just as important as functional requirements because they address system performance and scalability.

Examples:

- **Scalability:** Can the system handle an increasing number of users or transactions?
- **Latency:** How fast should the system respond to user requests?
- **Availability:** How often should the system be available? Is there an acceptable downtime window?
- **Security:** Are there specific measures in place for data protection, authentication, and authorization?

Key Points

- Include metrics that define system performance.

- Consider scalability, availability, reliability, and performance.
- Focus on how well the system can handle growth or high demand.

---

3. Hero Use Cases

Hero use cases, also known as **primary use cases**, define the most critical and frequent paths through the system. These are the scenarios where the system is used in its most straightforward and impactful manner.
Focusing on hero use cases ensures that the most essential parts of the system are well-defined and thoroughly considered in the design.

For Example:

- **For an e-commerce site:** The hero use case might be a user searching for a product and making a purchase.
- **For a parking lot system:** The hero use case could be parking a car and retrieving it when needed.

Key Points

- Identify the primary users and actions that occur most frequently.
- These use cases drive the core functionality of the system.
- These should be clear and error-free to avoid misunderstandings.

By clearly outlining and understanding these requirements, you ensure a solid foundation for the design process, helping you avoid costly mistakes and unnecessary features later in the interview.

Step 2: Identify Core Entities

Once you have clarified the requirements, the next step is to **identify the core entities** involved in the system. Core entities are the primary objects that drive the functionality and structure of the system. These are the building blocks of your design, and understanding them will guide you in designing the relationships and operations needed.

A. Define All Core Domain Entities Relevant to the System

The core entities are the key components that the system will manage and interact with. These entities directly affect how the system behaves and what data it processes. Each entity has its own lifecycle, attributes, and responsibilities within the system.

For Example:

- In a movie streaming system, core entities could include User, Movie, and Subscription.
- In a parking lot system, the core entities might be Car, ParkingSlot, and Ticket.

Key Points:

- **Identify** entities central to the problem.
- **Clarify** the role each entity plays in the system.
- **Avoid** introducing too many irrelevant entities at this stage.

B. Include Attributes, Responsibilities, and Relationships Between Entities

Once you identify the core entities, you need to define their **attributes**, **responsibilities**, and **relationships**.

- **Attributes:** These are the properties or characteristics of the entity. For example, a **User** might have attributes like **userID**, **name**, **email**, and **subscription status**.
- **Responsibilities:** These describe the tasks or operations the entity must perform. For a **Movie**, its responsibility might be to store information about its title, genre, and duration.
- **Relationships:** How entities interact with each other. For example, a User can have a Subscription, and a Movie might belong to a Genre.

Key Points:

- Attributes define the data associated with each entity.
- Responsibilities describe what each entity must be able to do.
- Relationships outline how entities work together or depend on each other.

C. Mention Auxiliary/Supporting Entities If Required

In addition to the core entities, there might be **auxiliary or supporting entities** that help manage auxiliary functionality or provide additional context. These entities are not central to the core functionality of the system, but they support the operations of core entities.

For Example:

- In a **movie streaming system**, supporting entities might include **Actor** or **Director**.
- In a **parking lot system**, supporting entities could be **ParkingFee** or **Payment**.

Step 3: Visualize Interaction Flow

After identifying the core entities, the next step is to **visualize the interaction flow**. This step is vital for understanding how different components of the system interact with each other and how users interact with the system. Visualizing the flow will help ensure that data moves correctly between components, services, and users, making the design process much clearer and more efficient.

A. Describe Who Interacts with Whom During Interactions

The first part of visualizing the interaction flow is to understand and describe who interacts with whom during the system's operation. This includes both internal system services and external systems interacting with each other, and most importantly, how users interact with the system. By identifying these relationships, you ensure that the system is designed to handle real-world scenarios appropriately.

Key Points:

- **Identify** the entities involved in each interaction (e.g., user, system services, external services).
- **Clarify** how data or requests flow between entities.
- **Determine** the sequence of actions that take place during an interaction.

B. Use Sequence Flows, Flowcharts to Show Interactions

Sequence flows and flowcharts are valuable tools to visually represent how the system interacts with users and services. These diagrams help illustrate the step-by-step flow of data or actions that take place during interactions. It is important to include both internal and external interactions. For example:

- **Internal Service Interactions:** How internal services communicate with one another within the system (e.g., service calls, internal processing).
- **External System Calls:** Illustrating interactions with external systems such as payment gateways, third-party services, or notifications systems. This includes any API calls or external communication initiated from the system.

Key Points:

- **Sequence flows** help show the order of operations and interactions.
- **Flowcharts** provide a clear, visual representation of system behavior.
- Both **internal** and **external interactions** must be well-defined to avoid miscommunication.

C. System Interactions with Users

It's essential to specifically define how the system will interact with users. This includes all the key user actions such as logging in, making a purchase, or submitting a form. Understanding the user's journey and how the system responds will help you design an intuitive and efficient user interface and experience.

Key Points:

- Identify all user actions that trigger system responses.
- Map out how the system should behave in response to user inputs.
- Ensure that the system feedback is clear and aligned with user expectations.

D. External System Calls

It’s also important to account for interactions with external systems that are crucial for your application’s functionality. These calls are essential for extending the system's capabilities, such as payments, notifications, and data exchanges. For example:

- **Payment Gateway:** The system will interact with external payment gateways to process transactions securely.
- **Notifications:** External systems might be used to send notifications (e.g., email or SMS) to users based on system events.

Key Points:

- Define all external systems the application will interact with.
- Ensure that the system handles these calls securely and efficiently.
- Account for any potential latency or failure in external calls and design fallback mechanisms.

Step 4: Define Class Structures and Relationships

The next crucial step in designing a system is to **define the class structures and relationships**. This involves determining the organization of the system's code and how various classes interact with one another. Properly structuring the classes and their relationships helps maintain a clean, maintainable, and scalable codebase. In this step, you'll leverage **OOP principles** and the **SOLID principles** to create a well-structured design.

A. Use OOP and SOLID Principles

Object-Oriented Programming (OOP) principles, combined with SOLID principles, ensure that the system is easy to scale, test, and maintain. The SOLID principles (Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion) guide how classes

should be structured and how they should interact with each other. For example:

- **Single Responsibility Principle (SRP):** Each class should have one responsibility or reason to change.
- **Open/Closed Principle (OCP):** Classes should be open for extension but closed for modification.
- **Liskov Substitution Principle (LSP):** Objects of a superclass should be replaceable with objects of its subclasses without affecting the system's correctness.

These principles help keep the system modular, flexible, and easy to maintain over time.

Key Points:

- **Leverage OOP principles** for structuring code and defining class responsibilities.
- **Apply SOLID principles** to ensure scalability and maintainability.

B. Include Interfaces, Abstract Classes, and Concrete Implementations

In the design, it’s essential to define the structure of each class, separating the interfaces, abstract classes, and concrete implementations. Interfaces and abstract classes define the contract and behavior of the classes, while concrete implementations provide the actual logic. For example:

- **Controller Layer:** This is where user requests are handled. The controller accepts requests, processes them, and passes them to the service layer for further processing.

- **Service Layer:** This layer contains the business logic of the system. It performs operations and manipulations based on the requests it receives from the controller.
- **Repository Layer:** This layer is responsible for handling data access logic. It interacts with databases or other data sources to fetch or persist data.
- **Domain Models:** These represent the core business objects and data structures in the system (e.g., User, Movie, Order).

These layers help to separate concerns and allow for easier testing, maintenance, and scalability.

Key Points:

- **Use interfaces** to define contracts between layers.
- **Define abstract classes** for shared behavior.
- **Concrete implementations** should provide the actual logic based on the defined interfaces/abstract classes.

C. Apply Design Principles for Scalability and Extensibility

As you define the class structures, it's important to apply design principles that ensure the system can scale and remain extensible. Two key principles here are **loose coupling** and **high cohesion**. These principles help reduce the interdependencies between different classes and ensure that each class focuses on a specific set of responsibilities.

- **Loose coupling** means that classes should not depend heavily on one another. Changes in one class should have minimal impact on others.

- **High cohesion** means that a class should be focused on a specific task or responsibility, making it easier to understand and maintain.

Additionally:

- Use abstractions for third-party integrations (e.g., payment gateways, external APIs).
- Ensure modularity by organizing code into smaller, manageable components that represent business logic.

Key Points:

- **Loose coupling** minimizes dependencies between classes.
- **High cohesion** ensures that classes have a single, well-defined responsibility.
- **Use abstractions** when dealing with third-party systems to prevent tight coupling with external services.

D. Prepare the Design for Production Scale

Finally, it's essential to prepare the design for production. This means considering how the system will perform under load, how it will handle failures, and how it will scale to accommodate more users or data. By designing with scalability in mind, you ensure the system can grow without significant rewrites.

Key Points:

- **Consider scalability** by planning how to handle increased traffic, larger datasets, and additional features.
- **Ensure fault tolerance** to handle failures gracefully and maintain system uptime.

- **Design for maintainability** to ensure that the system can be updated easily as requirements evolve.

Step 5: Define Core Use Cases and Methods

Once the system architecture and class structures are defined, the next step is to **define the core use cases and methods**. This step focuses on detailing the major features of the system by specifying their corresponding methods, responsibilities, and interactions. Properly defining use cases ensures that the system is functionally complete, with well-structured processes for each task it performs.

A. For Every Major Feature, Define

For each of the core features, define the essential aspects that guide how the system should function. This includes:

- **Method Responsibilities:** Every method in the system should have a clear responsibility. Define what each method does, what parameters it needs, and what it returns.
- **I/O Models:** Specify the input and output models used for interaction. This could include how data is received from the user and how it is returned or displayed.
- **Collaborating Classes:** Identify which classes need to collaborate for each feature to work. For example, the controller might collaborate with the service and repository layers to process requests and interact with the database.
- **Transaction Flow:** Define the transaction flow for key processes, such as how data moves through the layers and how different components interact during the execution of each feature.

Key Points:

- **Define method responsibilities** clearly for each function.

- **Identify I/O models** to manage user inputs and outputs effectively.
- **Clarify collaborations** between classes to understand the flow of data and processes.
- **Detail transaction flows** to visualize how data moves through the system during operations.

B. Include Use Cases Like

It's also important to include specific use cases that highlight real-world interactions. These use cases should cover common operations within the system and address how the system manages data and processes in typical scenarios. Examples include:

- **Create, Update, Delete, Fetch Operations:** These are basic CRUD operations that form the backbone of system functionality. Every system should be able to create, update, delete, and fetch data as required.
- **Real-Time Flows:** Some use cases require real-time operations, such as seat locking in booking systems, or updating order statuses in an e-commerce system. These should be handled with care to ensure timely and efficient execution.
- **Background Tasks:** Certain system tasks may run asynchronously in the background, such as processing payment transactions or sending emails. These tasks are typically not user-facing but are critical to maintaining the system's overall function.

Key Points:

- **Define CRUD operations** for managing system data.
- **Identify real-time flows** that require fast, synchronous updates.
- **Design background tasks** for asynchronous operations that do not block user interactions.

Step 6: Apply Design Patterns

Once the core use cases and methods are defined, the next step is to **apply design patterns** to your system design. Design patterns are proven solutions to common software design problems, which help ensure that your code is flexible, scalable, and maintainable. By choosing and applying the right design patterns, you can ensure that your system architecture adheres to best practices and is easy to extend or modify in the future.

A. Mention Clearly Which Design Patterns and Why

For each major component or feature in the system, choose appropriate design patterns to guide your solution. It's important to not only apply a design pattern but also to understand why you are using it. The right design pattern can improve system scalability, reduce complexity, and help make the system more maintainable.

Examples of common design patterns:

- **Singleton Pattern:** Useful when you need to ensure that a class has only one instance and provides a global point of access to that instance.
- **Factory Pattern:** Ideal for creating objects without specifying the exact class of object that will be created. This helps in decoupling the code from specific classes.
- **Observer Pattern:** Useful for designing a subscription mechanism where one object (the subject) notifies other objects (observers) about changes in its state.
- **Strategy Pattern:** Useful when you need to select one of many algorithms at runtime. It defines a family of algorithms, encapsulates each one, and makes them interchangeable.

Key Points:

- **Choose** design patterns that solve specific problems in your system.
- **Justify** the choice of each design pattern based on the system's needs and requirements.
- **Make patterns interchangeable** where possible to improve flexibility and scalability.

B. Reinforce Adherence to Clean Code Practices

Applying design patterns also means adhering to clean code practices. This involves writing code that is readable, maintainable, and easy to understand. By following clean code principles, you ensure that your system design is clear and that the codebase can be easily modified or extended in the future.

Clean code practices include:

- **Meaningful Naming:** Use descriptive names for classes, methods, and variables so that their purpose is immediately clear.
- **Consistent Formatting:** Follow a consistent code style for indentation, spacing, and line breaks.
- **Modularity:** Break down the system into small, independent components that are easy to test and maintain.
- **Documentation:** Document complex or critical parts of the system to ensure that future developers can easily understand them.

Key Points:

- **Follow clean code practices** to ensure maintainability and readability.
- **Refactor regularly** to keep the codebase clean and efficient.

- **Document code** where necessary, especially for complex or business-critical logic.

Step 7: Handle Edge Cases

Once the system's core functionality is established, the next step is to **handle edge cases**. Edge cases are scenarios that occur at the extremes of system functionality, often leading to unexpected behavior or failures. Addressing these edge cases ensures that the system remains stable, reliable, and resilient under a variety of conditions.

A. Discuss Edge Cases, Failure Scenarios, and System Limits

It's important to anticipate and address the various edge cases that can arise in the system. These include unusual or rare situations that may not occur often but can cause system failures if not handled properly. Common edge cases include:

- **Concurrency Issues:** Problems that occur when multiple processes or threads attempt to access shared resources at the same time. This can lead to race conditions, deadlocks, or data inconsistency.
- **State State:** Managing the state of the system can be complex, especially when dealing with partial updates or failures. It’s important to ensure that the system remains consistent even in the event of state inconsistencies.
- **Partial Failures:** A system may not always fail completely but could experience partial failures. For example, one part of the system may fail while others continue working. Handling partial failures gracefully ensures that the system remains available to users even when parts of it are down.
- **Retry Handling and Idempotency:** If an operation fails, the system should be able to retry the operation without causing additional

issues. Ensuring that operations are idempotent, meaning they can be retried without side effects, is critical for reliability.

Key Points:

- **Anticipate concurrency issues** and address them using synchronization mechanisms.
- **Ensure state consistency** even when failures occur.
- **Handle partial failures** gracefully to maintain system availability.
- **Implement retry mechanisms** and ensure idempotency for safe retries.

B. Discuss Mitigation Strategies

To deal with the challenges of edge cases, you should implement mitigation strategies that can handle or minimize the impact of failures. Common strategies include:

- **Locks, Cache Invalidation, Compensation:** These techniques help manage concurrency, ensure data consistency, and allow the system to recover from failures. For example, locks prevent multiple processes from modifying the same data simultaneously, while cache invalidation ensures that stale data is not used.
- **Consistency vs. Availability Trade-Offs:** Systems often face the challenge of balancing consistency and availability. For example, in a distributed system, maintaining strong consistency can reduce availability, while prioritizing availability may allow inconsistent data to be temporarily served. The trade-offs should be carefully considered based on the specific needs of the system.

Key Points:

- **Apply locks** to prevent conflicts in concurrent operations.
- **Use cache invalidation** to keep data up-to-date.

- **Handle compensation** to reverse partial operations in case of failures.
- **Balance consistency** with availability to optimize system performance.

Step 8: Class Diagram and Package Structure

After defining the core components and ensuring their functionality, the next step is to **design the class diagram** and **package structure**. This step helps in visualizing the static structure of the system, ensuring that the relationships between classes are clearly defined, and providing a roadmap for how different components will be organized into packages.

A. Class Diagram

The class diagram visually represents the system's classes, their attributes, methods, and the relationships between them. This diagram helps in understanding the overall architecture and ensuring that the system is modular and well-organized. Key relationships in the diagram include inheritance, composition, and associations between classes.

Key elements to include in the class diagram:

- **Classes:** Represent the key entities in the system.
- **Attributes:** Define the data held by each class.
- **Methods:** Represent the behavior of each class.
- **Relationships:** Show how the classes interact with each other (e.g., inheritance, association, aggregation).

Key Points:

- **Define clear relationships** between classes to avoid tight coupling.

- **Ensure methods** in classes follow single responsibility and other design principles.
- **Utilize UML notation** to represent classes and relationships clearly.

B. Package Structure

Organizing the system into packages helps in keeping the codebase modular and maintainable. Group related classes into packages based on their functionality, making the system easier to navigate and extend.

Consider organizing by:

- **Layered Structure:** Organize packages based on the layers of your system, such as the controller, service, repository, and domain layers.
- **Feature-Based Structure:** Organize packages by features or modules, such as user management, payment, etc.
- **Utility or Helper Packages:** Group utility classes or helpers into separate packages to promote reusability.

Key Points:

- **Follow modularization principles** to ensure code reusability and separation of concerns.
- **Ensure clarity** by naming packages in a way that reflects their functionality.

Step 9: Discuss Future Add-ons

The next step is to **discuss future add-ons** that can be added to the system. Planning for future enhancements or modifications ensures that the system remains flexible and extensible. By thinking ahead, you can design the system to accommodate new features without requiring a complete overhaul.

A. Identifying Potential Add-ons

When discussing future add-ons, it's important to identify potential features or functionalities that could be integrated into the system in the future. These could be driven by user feedback, market demands, or system performance requirements.

Examples of possible add-ons:

- **Integration with third-party services:** Adding payment gateways, social media authentication, or email notification systems.
- **Scalability Enhancements:** Adding load balancing, caching mechanisms, or improved database architectures for handling more traffic.
- **New Functionalities:** Adding features like search functionality, data analytics, or machine learning algorithms for predictions.

Key Points:

- **Plan for extensibility** by identifying future needs and ensuring the system can accommodate them.
- **Document potential add-ons** to keep stakeholders informed of possible future features.

B. Ensure Extensibility in Design

When designing the system, make sure that the architecture allows for future extensions with minimal effort. Using design patterns, modular architecture, and proper abstraction layers helps ensure that future additions do not disrupt the current functionality.

Key Points:

- **Use design patterns** like Factory, Singleton, or Observer to ensure extensibility.
- **Maintain clear separation of concerns** to make future modifications easier.
- **Test add-ons** to ensure they integrate seamlessly with the existing system.

FAQs and Common Doubts

As you approach your system design interview, it's common to have some lingering doubts. Addressing frequently asked questions (FAQs) can help clarify these and ensure you are fully prepared for the interview. Below are some common questions you might encounter and tips to handle them effectively:

1. How much detail should I go into?

While discussing your design, it's essential to strike a balance between providing enough detail and staying focused on the high-level design. Start by outlining the major components and their interactions, then drill down into the specifics only when necessary. Don't over-explain trivial details but ensure that crucial aspects are covered, such as data flow, class responsibilities, and key interactions.

Key Points:

- Provide **high-level overview** first and dive deeper when needed.

- **Don't get lost in unnecessary details** but be ready to answer any questions.

---

2. Should I write code during the discussion?

Writing code during the interview is generally not recommended unless explicitly asked to do so. Focus on the design discussion and high-level architecture first. Use pseudocode or diagrams to illustrate your ideas, and save actual coding for the implementation phase if requested. However, be ready to discuss how you would translate your design into code.

Key Points:

- **Focus on design** and **architecture** during the discussion.
- Use **pseudocode** to describe the algorithm and logic if needed.

---

3. How much business logic should I implement?

In a system design interview, your goal is to focus on the overall system architecture rather than implementing the business logic in detail. Discuss how you would handle key aspects of the logic, but don’t get bogged down in the minutiae. Acknowledge that business logic can be implemented later in the service layer and explain its role in the system.

Key Points:

- **Describe how** you would implement business logic in the service layer.
- **Focus on the system design** rather than writing detailed business logic.

---

4. Which patterns to use?

During system design, knowing which design patterns to apply is crucial. Certain patterns are more appropriate for specific problems. Below are some common patterns you can use:

- **Strategy Pattern:** Use for pricing or selection logic.
- **Factory Pattern:** Ideal for object creation.
- **Observer Pattern:** Use for event handling or notification systems.
- **Repository Pattern:** Great for handling data access operations.
- **Adapter Pattern:** Useful when integrating with third-party services.

Key Points:

- **Choose design patterns** based on the problem you are solving (e.g., pricing, event handling, data access).
- **Justify your pattern choice** based on system requirements and scalability.

---

5. What do interviewers judge you on?

Interviewers generally assess several key areas during a system design interview:

- **Handling Edge Cases:** Ability to identify and address edge cases and failure scenarios.
- **OOP, SOLID Principles:** Understanding and applying object-oriented principles and the SOLID design principles.
- **System Design:** How well you can structure and design the system architecture for scalability and performance.

Key Points:

- **Focus on designing a system** that can scale and handle edge cases.
- **Ensure your design follows OOP** and SOLID principles for maintainability.